Bagging, Random Forest, and Gradient Boosting per la classificazione

Lucrezia Tusetti

Abstract

Questo lavoro esplora l'applicazione di metodi di statistical learning basati su alberi decisionali per la classificazione di stelle, utilizzando il dataset "Star Dataset for Stellar Classification". L'obiettivo principale è distinguere tra stelle nane (Dwarf) e giganti (Giant). Il documento descrive come, partendo dai dati grezzi, siano state effettuate delle operazioni di pulizia e pre-processing dei dati, come la rimozione di osservazioni con elevato standard error e la semplificazione della variabile Sptype. Successivamente, vengono introdotti e confrontati tre algoritmi di apprendimento automatico: Bagging, Random Forest e Gradient Boosting. I risultati mostrano come Random Forest migliori leggermente l'accuratezza rispetto a Bagging, ma la differenza non è sostanziale. L'applicazione di Gradient Boosting porta ad un'accuratezza simile a quella di Random Forest. L'analisi evidenzia inoltre che l'errore di classificazione più comune è quello in cui le stelle Giganti vengono erroneamente classificate come Nane.

1.Introduzione

Le stelle sono sfere di gas incandescenti, che emettono luce e calore a causa delle reazioni nucleari che avvengono al loro interno. Queste reazioni liberano enormi quantità di energia sotto forma di radiazione elettromagnetica, che è percepita come luce. La classificazione delle stelle si basa su vari parametri come la loro temperatura, luminosità, composizione chimica e fase evolutiva. Il sistema di classificazione più utilizzato è quello di Harvard, introdotto nel 1901 da Annie Jump Cannon, che suddivide le stelle in diverse classi spettrali, ordinate in base alla temperatura della loro superficie.
La necessità di classificare le stelle nasce dalla volontà di comprendere meglio la loro natura, evoluzione e posizione nell'universo, e si è intensificata con l'evoluzione delle tecniche osservative, che oggi comprendono anche l'uso di metodi avanzati dello statistical learning per l'analisi dei dati osservati. Questi approcci hanno permesso di ottenere una classificazione più precisa e dettagliata delle stelle, considerando non solo le caratteristiche visibili, ma anche le informazioni derivanti da modelli computazionali più complessi.
Classificare le stelle è importante per diversi motivi. Innanzitutto, consente di studiare l'evoluzione stellare, dalle fasi iniziali di formazione a quelle finali di morte. Inoltre, aiuta a determinare la composizione chimica delle stelle e a capire come queste influenzino la formazione dei pianeti e la dinamica delle galassie.

Il dataset preso in esame nell'elaborato è "Star Dataset for Stellar Classification". Il dataset è composto da 6 variabili e dalla variabile *TargetClass* e contiene la descrizione di 3642 stelle. L'obiettivo del lavoro è quello di classificare le osservazioni come stelle *Dwarf* (nane) oppure *Giant* (giganti), utilizzando metodi basati sugli alberi decisionali.In particolare, verranno utilizzati e confrontati i metodi Bagging, Random Forest e Boosting.

```r
data <- read.csv("Star3642_balanced.csv")
data$TargetClass <- factor(data$TargetClass)
```

Variabile	Descrizione
Vmag	Magnitudine apparente visibile della stella
Plx	Distanza tra la stella e la Terra
e_Plx	Standard error di Plx

B-V	Indice colore B-V. Una stella calda ha un indice di colore B-V vicino a 0 o negativo, mentre una stella fredda indice B-V positivo
SpType	Tipologia Spettrale
Amag	Magnitudine assoluta della stella
TargetClass	1 se Dwarf, 0 se Giant

Come prima cosa, vengono eliminate le osservazioni che riportano uno standard error *e_Plx* molto elevato e che vengono pertanto ritenute incerte. La soglia utilizzata è la media della variabile stessa più due volte la sua deviazione standard.

```
media_se <- mean(data$e_Plx, na.rm = TRUE)
sd_se <- sd(data$e_Plx, na.rm = TRUE)

soglia_2sd <- media_se + 2 * sd_se
data <- data[data$e_Plx <= soglia_2sd,]
```

Un'ulteriore modifica riguarda la variabile *Sptype*, la quale è rappresentata da un codice alfanumerico. Per semplicità, si è deciso di considerare solo il primo elemento del codice, che identifica la classe spettrale della stella.

```
data$SpType <- substr(data$SpType, 1, 1)
data$SpType <- factor(data$SpType)
```

Classe_Spettrale	Descrizione
O	Stelle più calde e massicce, molto luminose.
B	Stelle calde e massicce, meno luminose delle O.
A	Stelle di temperatura media, spesso visibili ad occhio nudo.
F	Stelle di temperatura media, meno comuni.
G	Stelle simili al nostro Sole (es. il Sole è una G2V).
K	Stelle di bassa temperatura, più comuni di quanto si pensi.
M	Stelle più fredde e deboli, spesso nella fase finale della vita.

2. Metodi utilizzati

I metodi applicati sono bagging, Random Forest e Boosting, i quali si basano sull'aggregazione dei risultati restituiti da un ensamble di alberi di classificazione.

Nel **Bagging**, vengono entratti B campioni bootstrap Z^{*b} dal set di training e su ognuno di questi viene addestrato un albero di classificazione. Si ottengono quindi B classificatori individuali $\hat{G^{*b}}(x)$. La classe con il maggior numero di voti viene assegnata come previsore finale $\hat{G}_{bag}(x)$.
Il principale limite del metodo bagging è la correlazione tra gli alberi. Infatti, seppur ogni albero sia addestrato su un campioni differenti, bisogna tener conto del fatto che questi sono estratti da uno stesso training set e pertanto non sono indipendenti.

Il metodo delle **Random Forest** apporta una modifica sostanziale al Bagging. La costruzione di questo modello è la medesima spiegata in precedenza ma viene introdotta una modifica, che ha l'obiettivo di superare il problema della correlazione tra gli alberi.
Infatti, oltre a campionare le osservazioni, ad ogni split vengono campionate un certo numero m di covariate e lo split verrà fatto soltanto sulla base di quelle estratte.
L'effetto sulla riduzione della della correlazione, porta alla riduzione della varianza, che ora è resa molto vicina al suo minimo, ovvero $\frac{\sigma^2}{n}$.

Infine viena applicato il **Gradient Boosting**, il quale costruisce alberi in maniera sequenziale sfruttando l'informazione estratta dagli alberi precedenti. Questo algoritmo si basa sul combinare diversi weak learner e

l'idea è quella di ottenere un comitato di soluzioni che, per quanto deboli, se considerate nell'insieme, possano restituire una previsione affidabile.
L'algoritmo si compone dei seguenti passi:

1. Inizializzazione di $f_0(x) = argmin_\gamma \sum_{i=1}^N L(y_i, \gamma)$
2. Per $m = 1, ..., M$
   - Per $i = 1, ..., N$ si calcola $r_{im} = -\frac{\delta L(y_i, f_{m-1}(x_i))}{\delta f_{m-1}(x_i)}$
   - Si fitta un albero di regressione per stimare i residui r_{im}, ottenendo J_m regioni terminali
   - Per $j = 1, ..., J_m$ si calcola $\gamma_j = argmin_\gamma \sum_{x_i \in R_j} L(y_i, f_{m-1}(x_i) + \gamma)$
   - Si aggiorna $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_j I(x_i \in R_j)$
3. Si restituisce l'output $\hat{f}_{gbm} = \hat{f}_M$

3. Tree-based methods

Come prima cosa vengono applicati i metodi di Bagging e Random Forest.

Bagging

```r
library(randomForest)
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```r
p <- ncol(data)-1
n <- dim(data)[1]

set.seed(123)
index <- sample(1:n, round(n*0.7),replace=FALSE)
train <- data[index,]
test <- data[-index,]

### BAGGING

bagging_model <- randomForest(TargetClass ~ ., data = train,
                              ntree = 500, mtry = p)
bagging_model
```

```
##
## Call:
##  randomForest(formula = TargetClass ~ ., data = train, ntree = 500,      mtry = p)
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 6
##
##         OOB estimate of  error rate: 11.09%
## Confusion matrix:
##      0    1 class.error
## 0 1154  131   0.1019455
## 1  146 1067   0.1203627
```

```r
bagging_prev <- predict(bagging_model, newdata = test)
tab_bag <- table(bagging_prev, test$TargetClass)
```

Random Forest

```
### RANDOM FOREST (mtry < p)
set.seed(123)
rf_model <- randomForest(TargetClass ~ ., data = train,
                              ntree = 500, mtry = floor(sqrt(p)))
rf_model
```

```
##
## Call:
##  randomForest(formula = TargetClass ~ ., data = train, ntree = 500,      mtry = floor(sqrt(p)))
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 2
##
##         OOB estimate of  error rate: 10.85%
## Confusion matrix:
##      0    1 class.error
## 0 1164  121  0.09416342
## 1  150 1063  0.12366035
```

```
rf_prev <- predict(rf_model, newdata = test)
tab_rf <- table(rf_prev, test$TargetClass)
```

Il Random Forest viene applicato con l'obiettivo di superare i limiti dovuti alla correlazione degli alberi nel Bagging. Le performance sono effettivamente lievemente migliorate: l'accuracy passa da 0.885 nel Bagging a 0.893 nel Random Forest.
Si riscontra pertanto un miglioramento, ma risulta essere molto contenuto. Questo suggerisce che la correlazione tra gli alberi potrebbe non essere particolarmente diminuita.
Si procede pertanto con il confronto delle correlazioni tra gli alberi nei due metodi e, alla luce dei risultati ottenuti, non ci si aspetta una variazione significativa.

Correlazione

```
extract_tree_predictions <- function(model, data) {
  n_trees <- model$ntree
  n_samples <- nrow(data)
  tree_preds <- matrix(NA, nrow = n_samples, ncol = n_trees)
  for (i in 1:n_trees) {
    tree_preds[, i] <- predict(model, newdata = data, predict.all =
                                TRUE)$individual[, i]
  }
  return(tree_preds)
}


calculate_tree_correlations <- function(tree_preds) {
  n_trees <- ncol(tree_preds)
  correlations <- numeric()
  for (i in 1:(n_trees - 1)) {
    for (j in (i + 1):n_trees) {
      corr <- sum(tree_preds[, i] == tree_preds[, j]) / length(tree_preds[, i])
      correlations <- c(correlations, corr)
```

```r
    }
  }
  return(correlations)
}
```

```r
# Correlazione bagging
bagging_extract <- extract_tree_predictions(bagging_model, test)

bagging_correlations <- calculate_tree_correlations(bagging_extract)
a <- mean(bagging_correlations)

# Correlazione RF
rf_extract <- extract_tree_predictions(rf_model, test)

rf_correlations <- calculate_tree_correlations(rf_extract)
b <- mean(rf_correlations)

print(paste("Correlazione Bagging:", round(a,3)))
```

```
## [1] "Correlazione Bagging: 0.842"
```

```r
print(paste("Correlazione Random Forest:", round(b,3)))
```

```
## [1] "Correlazione Random Forest: 0.837"
```

Come previsto, la correlazione non è migliorata significativamente. Questo spiega perchè le performance del Random Forest non sono molto differenti.

Gradient Boosting

```r
# Creo il validation
ii <- sample(1:dim(train)[1], round(0.5*dim(train)[1]), replace=FALSE)
train <- train[ii,]
val <- train[-ii,]
```

```r
library(gbm)
```

```
## Loaded gbm 2.2.2
```

```
## This version of gbm is no longer under development. Consider transitioning to gbm3, https://github.c
```

```r
set.seed(1)

# TUNING DEL LEARNING RATE E NUMERO ALBERI

hyper_grid <- expand.grid(
  learning_rate = c(0.05, 0.005),
  trees = c(250, 500),
  accuracy = NA
)

for(i in seq_len(nrow(hyper_grid))) {

  set.seed(123)
  m <- gbm(
    formula = TargetClass ~ .,
    data = train,
```

```r
    distribution = "bernoulli",
    n.trees = hyper_grid$trees[i],
    shrinkage = hyper_grid$learning_rate[i],
    interaction.depth = 3,
    n.minobsinnode = 10,
    cv.folds = 0
  )

  # Previsioni sul validation set
  pr <- predict(m, newdata = val, n.tree=500, type = "response")
  y.hat <- ifelse(pr > 0.5, 1, 0)

  # Accuracy
  actual_classes <- val$TargetClass
  hyper_grid$accuracy[i] <- mean(y.hat == actual_classes)
}

print(hyper_grid)
```

```
##   learning_rate trees  accuracy
## 1         0.050   250 0.9104235
## 2         0.005   250 0.8859935
## 3         0.050   500 0.9315961
## 4         0.005   500 0.8941368
```

Si sceglie come miglior learning rate $\lambda = 0.05$ e numero di alberi pari a 500. Si prosegue il tuning son la scelta dei migliori valori per *interaction.depth* e *n.minobsinnode*.

```r
set.seed(2)

hyper_grid <- expand.grid(
  interaction.depth = c(3, 5),
  n.minobsinnode = c(5, 10),
  accuracy = NA,
  trees=NA
)

for(i in seq_len(nrow(hyper_grid))) {

  set.seed(123)
  m <- gbm(
    formula = TargetClass ~ .,
    data = train,
    distribution = "bernoulli",
    n.trees = 500,
    shrinkage = 0.05,
    interaction.depth = hyper_grid$interaction.depth[i],
    n.minobsinnode = hyper_grid$n.minobsinnode[i],
    cv.folds = 10
  )

  # Numero alberi
  best_trees <- which.min(m$cv.error)
  hyper_grid$trees[i] <- best_trees
```

```r
  # Previsioni sul validation set
  pr <- predict(m, newdata = val, n.tree=best_trees,type = "response")
  y.hat <- ifelse(pr > 0.5, 1, 0)

  # Accuracy
  actual_classes <- val$TargetClass
  hyper_grid$accuracy[i] <- mean(y.hat == actual_classes)
}

print(hyper_grid)
```

```
##   interaction.depth n.minobsinnode  accuracy trees
## 1                 3              5 0.9055375   168
## 2                 5              5 0.9039088    99
## 3                 3             10 0.9055375   154
## 4                 5             10 0.9055375   108
```

Non si riscontrano variazioni sensibili. Si addestra il boosting considerando i parametri *interaction.depth = 3*, *n.minobsinnode = 10*.

```r
# BOOSTING

set.seed(123)
best_boosting <- gbm(
  formula = TargetClass ~ ., data = train, distribution = "bernoulli",
  n.trees = 500, shrinkage = 0.05,
  interaction.depth = 3, n.minobsinnode = 10,
  cv.folds = 10)

prob <- predict(best_boosting, newdata=test, type="response")
```

```
## Using 154 trees...
```

```r
y.hat <- ifelse(prob>.5, 1, 0)
tab_boost <- table(y.hat, test$TargetClass)
tab_boost
```

```
##
## y.hat   0   1
##     0 476  66
##     1  52 476
```

```r
round((tab_boost[1,1]+tab_boost[2,2])/sum(tab_boost),3)
```

```
## [1] 0.89
```

L'accuracy con il Gradient Boosting raggiunge il valore 0.89, molto simile al valore 0.893, ottenuto con Random Forest.
Per quanto riguarda l'accuracy, si può quindi concludere che le performance per i metodi applicati siano pressoché uguali.

```r
tab_boost; tab_rf; tab_bag
```

```
##
## y.hat   0   1
##     0 476  66
##     1  52 476
```

```
##
## rf_prev   0   1
##       0 481  67
##       1  47 475
```

```
##
## bagging_prev   0   1
##            0 476  71
##            1  52 471
```

Si può notare come gli errori più comuni sono quelli in cui la classe 1 viene prevista come 0. Le stelle di classe 1 sono le Giant, le quali vengono erroneamente classificate come Dwarf
Meno spesso, invece, stelle Dwarf vengono classificate come Giant.

Una possibile spiegazione del risultato ottenuto è che le stelle Giganti e Nane condividano alcune caratteristiche che potrebbero confondere il modello di classificazione. Ad esempio, le Giganti, essendo stelle più evolute e grandi, sono generalmente molto luminose, ma alcune di esse, in fase di evoluzione avanzata (come le Giganti rosse), potrebbero avere luminosità più bassa, simile a quella delle Dwarf.

4. Conclusione

In conclusione, questo studio ha applicato diversi metodi basati su alberi decisionali per la classificazione di stelle, con l'obiettivo di distinguere tra stelle nane e giganti. I risultati ottenuti con Bagging, Random Forest e Gradient Boosting mostrano performance simili in termini di accuratezza, con un leggero miglioramento di Random Forest rispetto a Bagging, dovuto all'introduzione di una fase di sotto-campionamento delle covariate ad ogni split, che diminuisce la correlazione tra gli alberi. Tuttavia, questo miglioramento non risulta significativo.
L'analisi degli errori di classificazione rivela che le stelle giganti sono spesso erroneamente classificate come nane, suggerendo una possibile sovrapposizione di caratteristiche tra le due classi. Questi risultati evidenziano la necessità di ulteriori approfondimenti per migliorare la classificazione e comprendere le caratteristiche che portano a tali errori. Si potrebbe ad esempio, applicare metodi diversi di classificazione, oppure considerare ulteriori variabili esplicative.